

Pre-Lecture Assignment 2

535510 Reinforcement Learning

Q1: Non-uniform Polyak-Łojasiewicz (PL) Condition

Lemma 8 states:

$$\left\| \frac{\partial V^{\pi_\theta}(\mu)}{\partial \theta} \right\|_2 \geq \frac{\min_s \pi_\theta(a^*(s) | s)}{\sqrt{S} \cdot \|d_\rho^{\pi^*} / d_\mu^{\pi_\theta}\|_\infty} \cdot [V^*(\rho) - V^{\pi_\theta}(\rho)]$$

(a) Why “non-uniform”?

In the standard PL condition $\|\nabla f\|^2 \geq \mu \cdot (f^* - f(\theta))$, the constant μ is fixed and does not depend on θ . In Lemma 8, the role of μ is played by:

$$\frac{\min_s \pi_\theta(a^*(s) | s)}{\sqrt{S} \cdot \|d_\rho^{\pi^*} / d_\mu^{\pi_\theta}\|_\infty}$$

This coefficient contains π_θ , which **changes as θ is updated during training**. Since the “PL constant” varies with the current policy rather than being uniform across all θ , this is called the **non-uniform** PL condition.

(b) Where does $\min_s \pi_\theta(a^*(s) | s)$ come from?

After substituting Lemma 1 (softmax policy gradient) into the gradient norm lower bound, we obtain:

$$\left\| \frac{\partial V^{\pi_\theta}}{\partial \theta} \right\|_2 \geq \frac{1}{\sqrt{S}(1-\gamma)} \sum_s d_\mu^{\pi_\theta}(s) \cdot \pi_\theta(a^*(s) | s) \cdot A^{\pi_\theta}(s, a^*(s))$$

Since $\pi_\theta(a^*(s) | s)$ differs across states, we factor it out using a **conservative lower bound** (taking the minimum over all states), ensuring the $\geq$ inequality holds:

$$\pi_\theta(a^*(s) | s) \geq \min_{s'} \pi_\theta(a^*(s') | s') \quad \forall s$$

This is where $\min_s \pi_\theta(a^*(s) | s)$ arises.

(c) Difference between μ and ρ

μ is the initial state distribution used **during training**. It satisfies $\mu(s) > 0$ for all s (Assumption 2), ensuring every state can be visited during rollouts for gradient updates.

ρ is the initial state distribution for **evaluation**, representing what we truly care about optimizing: $V^{\pi_\theta}(\rho) = \mathbb{E}_{s \sim \rho}[V^{\pi_\theta}(s)]$. The term $\|d_\rho^{\pi^*} / d_\mu^{\pi_\theta}\|_\infty$ captures the mismatch between the two distributions — the larger it is, the harder convergence becomes.

Q2: Q-value Actor-Critic (QAC) Algorithm

(a) Math equations for Lines 100–119

Given transition (s, a, r, s') , the update proceeds as follows:

Step 1: Compute current Q value

$$Q_w(s, a)$$

Step 2: Compute TD target

$$y = \begin{cases} r & \text{if done} \\ r + \gamma \sum_{a'} \pi_\theta(a' | s') \cdot Q_w(s', a') & \text{otherwise} \end{cases}$$

The expected next Q is computed under the **current policy** π_θ , with no gradient tracked (`torch.no_grad()`).

Step 3: Critic update — minimize TD error via MSE:

$$\mathcal{L}_{\text{critic}} = (Q_w(s, a) - y)^2, \quad w \leftarrow w - \alpha_w \nabla_w \mathcal{L}_{\text{critic}}$$

Step 4: Actor update — policy gradient weighted by Q value:

$$\mathcal{L}_{\text{actor}} = -\log \pi_\theta(a | s) \cdot Q_w(s, a), \quad \theta \leftarrow \theta - \alpha_\theta \nabla_\theta \mathcal{L}_{\text{actor}}$$

$Q_w(s, a)$ is detached so gradients do not flow into the critic during the actor update.

(b) Do you agree with GPT’s design in Line 112?

No. The standard QAC algorithm (Lecture 6) uses a **SARSA-style** target with a sampled next action $a_{t+1} \sim \pi_\theta$:

$$y_t = r_{t+1} + \gamma Q_w(s_{t+1}, a_{t+1}), \quad a_{t+1} \sim \pi_\theta(\cdot | s_{t+1})$$

GPT uses **Expected SARSA** instead — a policy-weighted expectation over all next actions:

$$y_t = r_{t+1} + \gamma \sum_{a'} \pi_\theta(a' | s_{t+1}) \cdot Q_w(s_{t+1}, a')$$

Both are on-policy, but Expected SARSA deviates from the standard QAC definition in the lecture slides.